COMPLETE PROJECT GUIDE

Self-Learning AI Agent

A Beginner's Guide to an AI That Learns From Experience

What the project is, how the agent learns, how we prove it, and what every part of the code, database and dashboard does — explained for someone who has never studied AI.



TECHNOLOGY

Python · PyTorch · Gymnasium · Streamlit · Plotly · SQLite

DEPLOYMENT

Web dashboard built with Streamlit, deployed on Streamlit Community Cloud

TEAM

Brahmanand Mathpati · Atharv Gundale · Vedant Biradar · Somesh Badwane

Final-year B.Tech CSE (AI Specialization), 2023–2027

LINKS

github.com/brahmanandmathpati/Self-Learning-AI-Agent

Live app: self-learning-ai-agent-final-year-project.streamlit.app

Document by **Brahms**

Apache-2.0 License

CONTENTS

Table of contents

Read it in order the first time. Every technical word is explained the first time it appears, and again in the Glossary (page 21).

1	Cover page	1
2	Table of contents	2
3	Problem statement	3
4	Objectives	3
5	Why we chose this project	4
6	What is Reinforcement Learning?	4
7	How the agent learns	5
7b	How DQN works	6
8	The two environments	7
9	System architecture	8
10	Backend explained	9
11	Frontend explained	11
12	Database explained	12
13	Complete working flow	13
14	Evaluation & proof	14
15	Ablation study	15
16	AI Reflection	16
17	Project file structure	17
18	How to run it	18
19	Testing & quality	18
20	Challenges & solutions	19
21	Limitations	19
22	Future scope & add-ons	20
23	Glossary	21
24	Conclusion & quick summary	22
25	Viva questions & answers	23

ONE ANALOGY FOR THE WHOLE GUIDE

Think of a **school**. The **agent** is the student. The **environment** is the exam. The **reward** is the marks. The **trainer** is the teacher. The **database** is the record book. The **dashboard** is the notice board where results are pinned.

SECTION 3 & 4

03 Problem statement & objectives

Many projects called “AI agents” do not actually learn. This project builds one that does — and proves it.

3 · The problem

What most “AI agents” do

They send a fixed instruction (a **prompt** — the text you type to a chatbot) to a large language model. The model answers from what it already knows. On day 1 and on day 100 it behaves the same way. Chat history or better wording is **not learning**.

What real learning needs

The agent must **change its own behaviour** because of experience: try something, see the result, and do better next time. And we must **prove** the improvement with fair tests, not just a nice-looking chart.

THE QUESTION THIS PROJECT ANSWERS

Can a program that starts with **zero knowledge** learn a task only from rewards — and can we **prove** it learned, with numbers that are fair and repeatable?

4 · Objectives

#	OBJECTIVE	WHERE IT IS DONE
1	Build an agent that learns from **rewards** (Reinforcement Learning)	Q-Learning, DQN
2	Learn two different tasks: a grid puzzle and a balancing task	FrozenLake, CartPole
3	Write the deep learning agent **from scratch** (no ready-made RL library)	PyTorch DQN
4	Test fairly: learning off, new games, 5 seeds, against a random player	Evaluation module
5	Use statistics to show the gain is not luck	Welch's t-test, bootstrap CI
6	Find which DQN parts really matter	Ablation study
7	Record every run so results can be compared and repeated	SQLite database
8	Explain results in plain words without inventing numbers	Reflection layer
9	Show everything in an easy, public website	Streamlit dashboard

SECTION 5 & 6

05 Why this project & what is RL

We wanted a project where learning is **real, measurable and explainable** — and that runs on a normal laptop.

Measurable

Learning shows up as a number that goes up — steps balanced, goals reached.

Fair to test

Games can be replayed with the same starting set-up, so before/after comparisons are exact.

Real AI skills

Neural networks, data, statistics, databases, testing and a web app — the full engineering stack.

Laptop friendly

Everything runs on a normal CPU. No graphics card (GPU) and no paid cloud service is needed.

Explainable

We can show exactly what changed inside the agent and why.

Honest

Every number on the dashboard comes from a real stored run.

6 · What is Reinforcement Learning?

Reinforcement Learning (RL) is a way of teaching a computer by giving it **rewards** for good actions. Nobody tells it the right answer. It tries, gets points, and slowly prefers the actions that earned points.

ANALOGY: TRAINING A PUPPY

The puppy sits → it gets a treat. Next time it sits more often. The puppy is the **agent**, the room is the **environment**, the treat is the **reward**. After many tries the puppy has a **policy** — a habit for what to do in each situation.

WORD	MEANING IN ONE LINE
Agent	The learner that makes decisions.
Environment	The world (game) the agent acts in.
State	What the agent sees right now.
Action	A move the agent can make.
Reward	Points the environment gives after a move.
Episode	One full game from start to end.



SECTION 7

07 How the agent learns

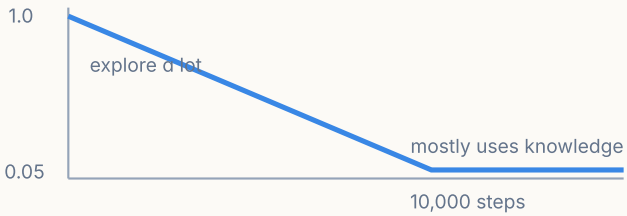
Learning happens in a small loop that repeats thousands of times. Each full game is one **episode**.

The learning loop

1. **Observe**
   The agent looks at the **state** (where it is).
2. **Choose**
   It picks an action. Sometimes it picks randomly to **explore**.
3. **Act**
   The game moves to a new state.
4. **Reward**
   The game gives points (or zero).
5. **Learn**
   The agent updates its memory of how good that action was.
6. **Repeat**
   Until the game ends; then a new episode starts.

Exploration (ε-greedy)

ε (epsilon) is the chance of making a random move. It starts at **1.0** (always random — the agent knows nothing) and falls to **0.05** over 10,000 steps. Early on the agent explores; later it mostly uses what it learned.



Q-Learning: the score table

Q-Learning keeps a table with one row per situation and one column per move. Each cell is a **Q-value** — "how good is this move here?". All cells start at 0. After each move the cell is updated:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r + \gamma \cdot \max Q(s', a') - Q(s, a) \,]$$

In plain words

new score = old score + a small step towards (reward now + best score expected next). **α = 0.1** is the step size (learning rate); **γ = 0.99** says how much future rewards count. At the end of a game there is no "next", so the target is just r.

Worked example

The agent stands next to GOAL and moves right. It reaches GOAL: r = 1, the game ends.
new = 0 + 0.1 × (1 − 0) = 0.1
"Move right" here now scores 0.1, better than 0 — so next time the agent prefers it. Over 2,000 episodes the good path spreads backwards to START.

SECTION 7 (CONTINUED)

7b How DQN works

CartPole has endless situations, so a table is impossible. A **Deep Q-Network (DQN)** uses a neural network to estimate the scores instead.

Neural network

A function with many adjustable numbers (**weights**) that learns from examples. Ours takes 4 numbers in, has two hidden layers of 128, and gives 2 scores out — one per action (4 → 128 → 128 → 2).

Replay memory

A store of the last 50,000 moves. The network learns from random batches of 64 old moves, not only the newest one. This makes learning steadier, like revising old lessons.

Target network

A frozen copy of the network that gives stable targets. It is refreshed every 500 learning updates. Without it the network chases its own changing guesses.

Exploration

Same ε-greedy rule as Q-Learning: 1.0 → 0.05 over the first 10,000 steps.

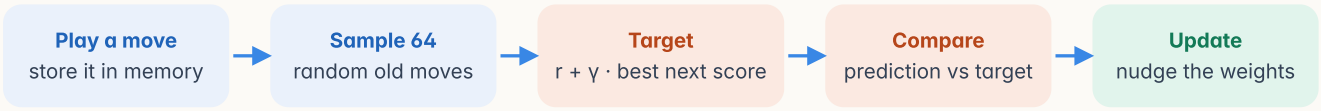
Huber loss

A way to measure "how wrong was the prediction" that does not panic over very large errors.

Adam + gradient clipping

Adam is the method that adjusts the weights; clipping stops any single update from being too big.

One DQN learning step



target comes from the frozen target network · learning starts after 1,000 moves are stored

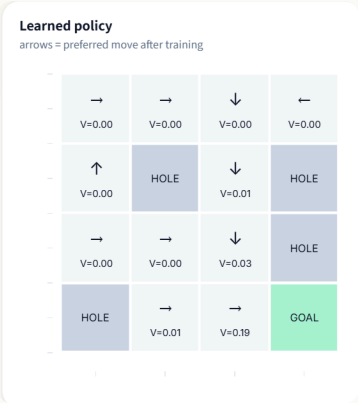
WHERE DOES THE LEARNING LIVE?

Only in **numbers**: the Q-table (FrozenLake) or the network weights (CartPole). After training, those numbers are saved in a **checkpoint** file — the agent's "brain on disk".

SECTION 8

08 The two environments

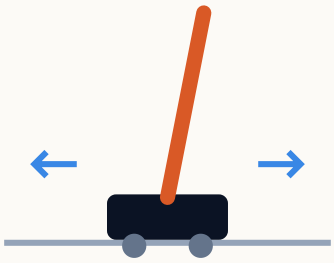
The agent practises in two standard training games from **Gymnasium**, a free library of simulated tasks.



Learned route on the dashboard

FrozenLake grid puzzle Q-Learning

Story	Walk across a frozen lake from START to GOAL without falling into holes.
Agent sees	Which of 16 squares it is on.
Actions	4 — left, down, right, up.
Reward	1 at the goal, 0 otherwise. Ends at the goal, in a hole, or after 100 steps.
Setting	Non-slippery by default (moves go where intended); slippery is optional.
Training	2,000 episodes, α = 0.1, γ = 0.99.



A pole on a moving cart

CartPole balancing DQN

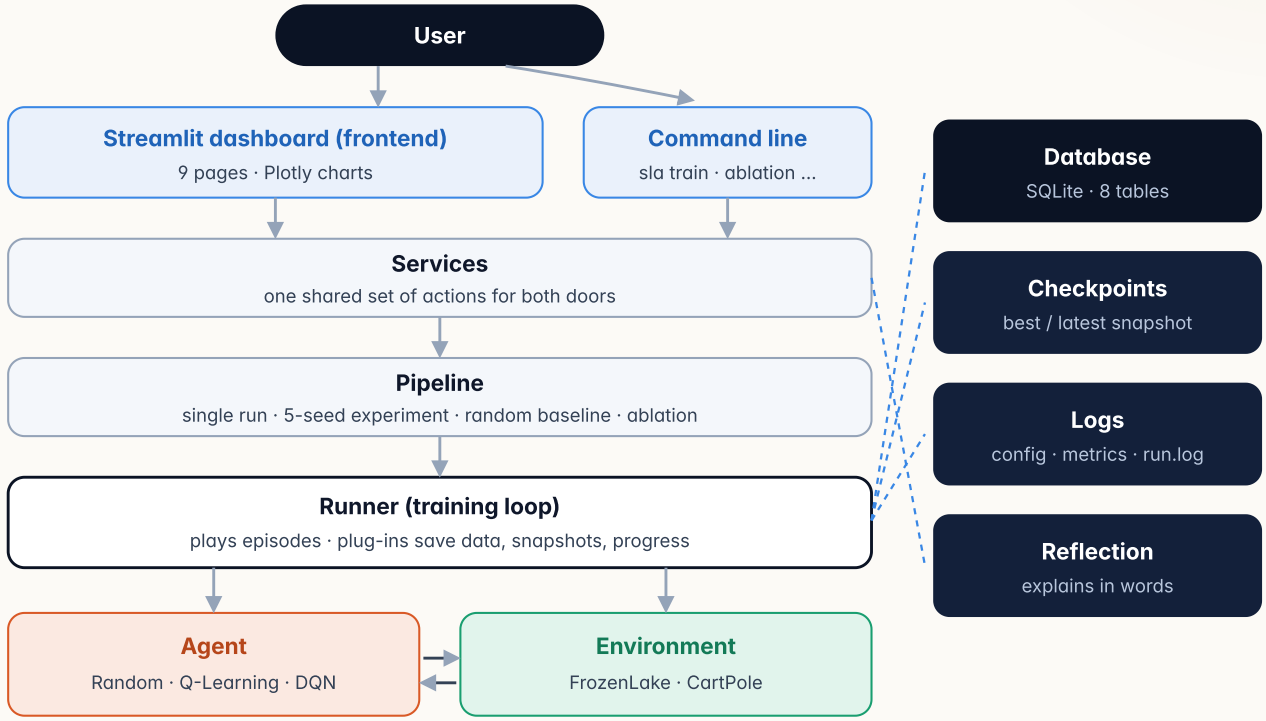
Story	Keep a pole upright on a cart that slides left and right.
Agent sees	4 numbers: cart position, cart speed, pole angle, how fast the pole falls.
Actions	2 — push left or push right.
Reward	+1 for every step the pole stays up.
Best score	500 steps (time limit). Reaching it is success, not failure.
Training	600 episodes of the DQN.

	FROZENLAKE	CARTPOLE
Kind of input	One of 16 squares (small, countable)	4 real numbers (endless combinations)
Learner	Q-table (16 × 4 numbers)	Neural network (~17,000 weights)
Random player scores	1.2% goals	22.6 steps

SECTION 9

09 System architecture

The project is built in **layers**. Each layer has one job and talks only to the layer below it.



Frontend

The part people see and click: the Streamlit website.

Backend

The part that does the work behind the scenes: training, testing, saving.

Services

A middle layer so the website and the command line use exactly the same code.

Pipeline

Runs complete jobs, e.g. “train 5 seeds, then test, then compute statistics”.

Runner

Plays the episodes. The only place where learning happens.

Side systems

Store results, snapshots and logs, and write explanations.

SECTION 10

10 Backend explained (1/2)

All program code lives in `src/sla/`. Each folder is one part of the system.

envs/ · the games

`factory.py`	Creates FrozenLake and CartPole from Gymnasium.

agents/ · the learners

`base.py`	Common template: act, learn, save, load.
`random_agent.py`	Random moves — the “no skill” comparison.
`q_learning.py`	Q-Learning with its score table.
`dqn.py`	Deep Q-Network: replay, target net, updates.
`networks.py`	The neural network 4 → 128 → 128 → 2.
`schedules.py`	ε going down from 1.0 to 0.05.

memory/

`replay_buffer.py`	DQN's memory of past moves; gives random batches.

training/ · the teacher

`config.py`	Reads and checks the YAML settings.
`runner.py`	The training loop — plays and learns.
`callbacks.py`	Plug-ins: save to DB, live progress, first test.
`guards.py`	Stops if numbers explode; warns if scores drop.
`safety.py`	Time limits and input protection.
`pipeline.py`	Full jobs: run, resume, experiment, baseline.

evaluation/ · the exam checker

`evaluate.py`	Tests the frozen agent (ε = 0) on new games.
`metrics.py`	Mean, median, success rate.
`stats.py`	Welch's t-test and bootstrap CI.
`ablation.py`	Full DQN vs no replay vs no target.
`plots.py`	Saves charts as images.

checkpoints/

`manager.py`	Saves/loads snapshots (best, latest) to resume.

SECTION 10 (CONTINUED)

10 Backend explained (2/2)

The rest of the backend stores data, writes explanations and gives the website and command line one shared entry point.

database/

`store.py`	SQLite: creates the 8 tables, saves and reads all results.

reflection/ · the explainer

`stats.py`	Collects facts (rewards, scores, p-value...).
`fallback.py`	Template that turns facts into sentences.
`llm_client.py`	Talks to Ollama, the optional local AI.
`grounding.py`	Checks every number against the facts.
`reflect.py`	Runs: facts → write → check → save.
`feedback.py`	Saves people's ratings of notes.

services/ · shared actions

`training_service.py`	Start runs and experiments.
`evaluation_service.py`	Evaluate a saved run.
`experiment_service.py`	Read results for pages.
`reflection_service.py`	Generate and rate notes.
`checkpoint_service.py`	List saved snapshots.

utils/ · helpers

`errors.py`	Project's own error types.
`io_helpers.py`	Read/write JSON and YAML.
`logging_setup.py`	Writes run.log.
`seeding.py`	Fixes randomness so runs repeat.
`summary.py`	Moving-average helper.
`validation.py`	Checks seeds and game steps.

Top-level files

`cli.py`	The **sla** command (train, evaluate, pipeline, ablation, baseline, reflect, runs, dashboard).
`__main__.py`	Allows `python -m sla`.
`settings.py`	Where the database and run folders are.

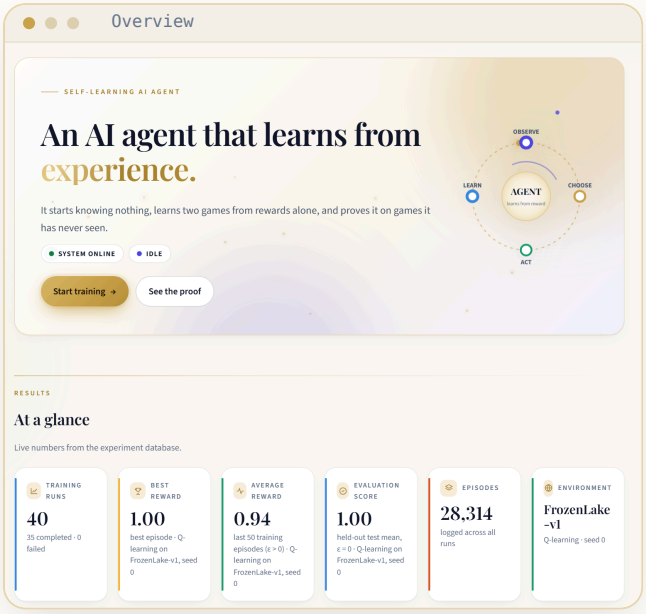
COMMAND-LINE EXAMPLES

```
sla train --config configs/frozenlak
e_qlearning.yaml
sla pipeline --config configs/cartpo
le_dqn.yaml --seeds 0 1 2 3 4
sla ablation --config configs/cartpo
le_dqn.yaml
sla dashboard
```

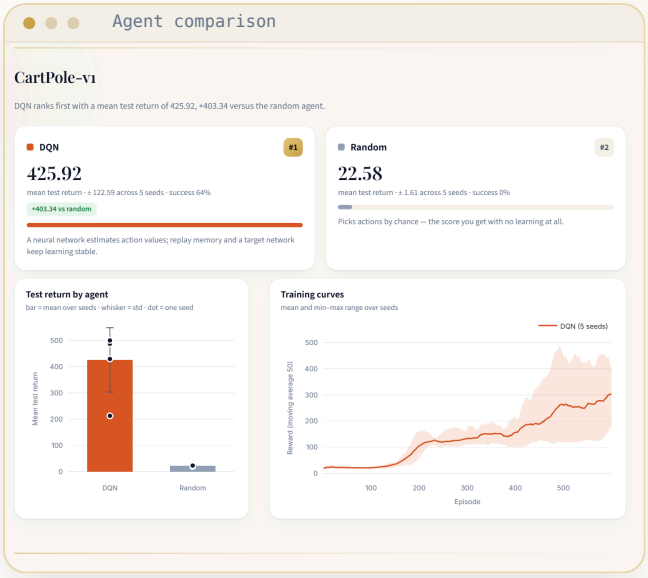
SECTION 11

11 Frontend explained

The website is built with **Streamlit** (turns Python into web pages) and **Plotly** (interactive charts). It lives in `src/sla/app/`.



Overview — hero with the animated learning loop, then live result cards.



Agent comparison — ranked cards and charts.

PAGE	WHAT YOU SEE	WHAT IT MEANS
Overview	Hero, how it works, features, result cards, featured run	Quick tour and health check
Training	Form + live progress, cards and charts	Start learning and watch it happen
Agent comparison	Ranked cards: trained vs random	Which agent is best and by how much
Evaluation	LEARNING VERIFIED banner, statistics	The proof that learning is real
Ablation	Full DQN vs no replay vs no target	Which DQN parts matter
Run explorer	One run's curves, settings, tests, logs	Full history of a single run
Reflection	Learning insight + data sources	Explanation in words, fact-checked
Environments	Game cards and the learned map	What the agent sees and does
About	Goal, architecture, team cards	Background information

Frontend files: `main.py` (starts the site, sidebar, error screens) · `views/` (9 pages) · `components/` (cards, badges, landing blocks, motion graphics) · `charts/figures.py` · `styles/theme.py` (ivory, ink and gold "luxury" theme + CSS motion) · `data.py` (fast cached reads) · `demo_seed.py` (loads the results snapshot on the live site).

SECTION 12

12 Database explained

A **database** is an organised record book. We use **SQLite**: the whole database is one file, with no separate server to install.

Why SQLite?

- Free, built into Python, zero setup.
- One file (`runs/sla.db`) — easy to copy and back up.
- Fast enough for tens of thousands of rows.
- Lets us **compare runs** instead of relying on memory or screenshots.

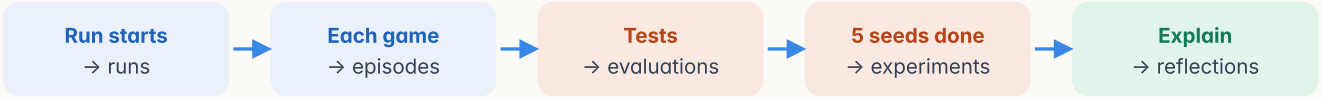
OUR REAL DATA

3 experiments · **36** runs · **20,314** episodes · 344 evaluations · 15 reflection notes. The live site loads a read-only copy of this (`data/demo/sla_demo.db`).

The 8 tables

`experiments`	Groups of runs (e.g. "DQN on CartPole, 5 seeds") and their statistics.	`runs`	One training run: game, method, seed, settings, status, times.
`episodes`	One row per game played: reward, length, ε, loss.	`metrics`	Extra named numbers, e.g. final test score.
`evaluations`	Tests: before training, during training, final test.	`ablation_results`	Score of each DQN variant per seed.
`reflections`	Written notes, the facts used, pass/fail of the check.	`feedback`	Ratings: accurate yes/no, usefulness 1–5.

How data flows into the tables



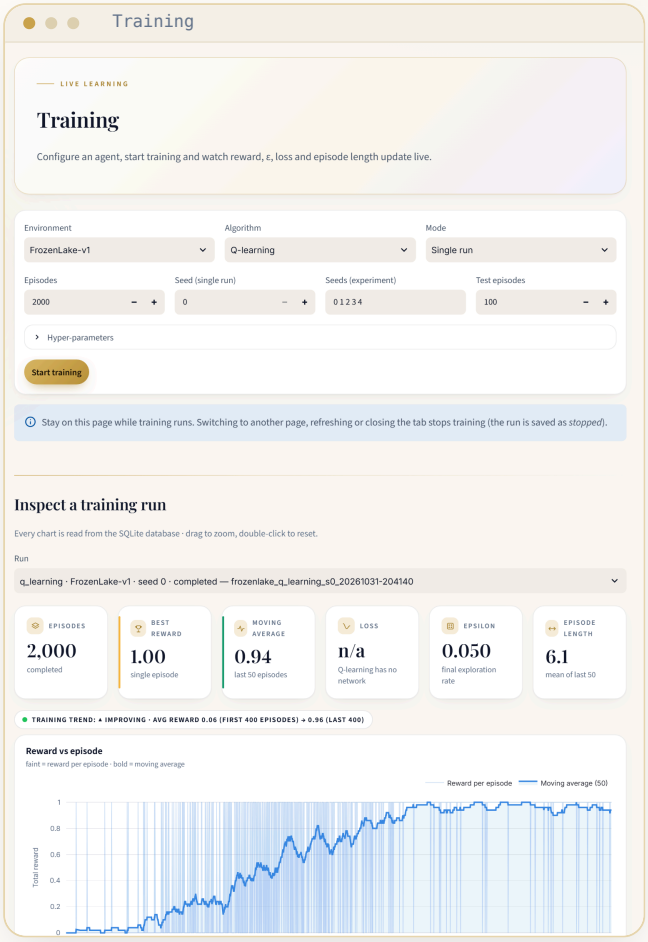
Every run also gets a folder `runs/<run_id>/` with `config.yaml` (settings), `metrics.json` (results), `run.log` (diary) and `checkpoints/` (saved brains).

SECTION 13

13 Complete working flow

What happens, step by step, from clicking **Start training** to seeing the proof on screen.

1. **You click Start training**
   On the Training page you choose the game, mode and settings.
2. **Settings are loaded**
   The service reads the YAML file in configs/ and checks it.
3. **A run is created**
   The pipeline gives it an ID and records it in the database.
4. **First test (before)**
   The untrained agent plays the test games — the “before” score.
5. **Training loop runs**
   The runner plays episodes; the agent learns after every move.
6. **Everything is saved**
   Each episode goes to the database; snapshots go to checkpoints/.
7. **Progress checks**
   Every few episodes a frozen test picks the best snapshot.
8. **Final test (after)**
   The best snapshot plays 100 new games with learning off.
9. **Experiment statistics**
   For 5 seeds: random baseline, t-test and confidence interval.
10. **You see the results**
    The dashboard reads the database and shows cards, charts and the verdict.



Training page — the form and a finished run's cards and charts.

SECTION 14

14 Evaluation & proof of learning

We never trust training scores alone: during training the agent still makes random exploring moves. So we run a separate, fair exam.

1 · Freeze

Learning off and random moves off (ε = 0). A copy is tested, so the real agent is untouched.

2 · New games

100 test games with set-ups (seeds) never used in training.

3 · Compare

Untrained vs trained vs random — on the same games, repeated with 5 seeds.

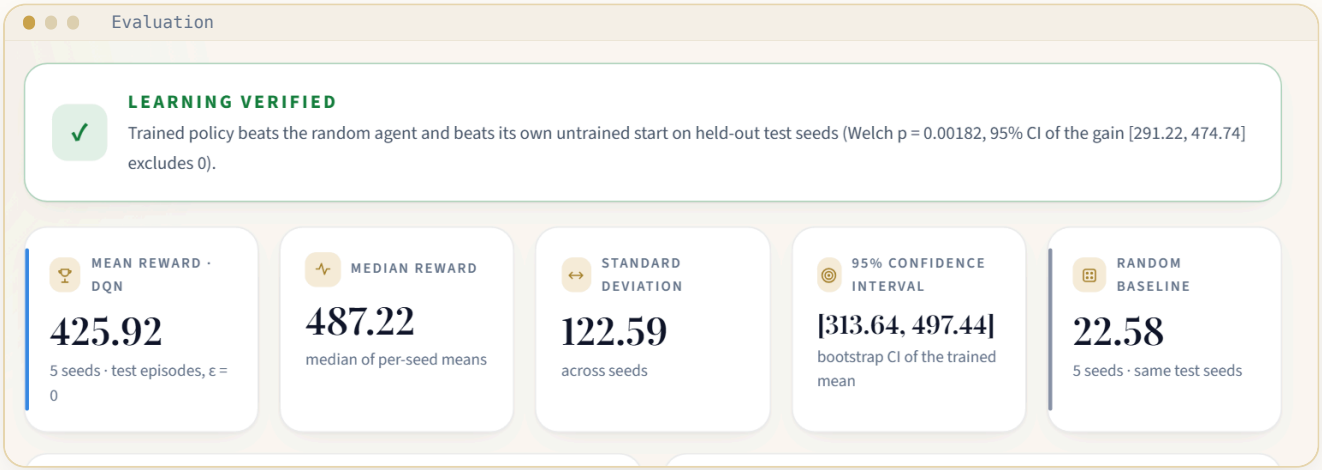
- **Seed** — a number that fixes the randomness, so a run can be repeated exactly.
- **Random baseline** — the score of a player with no skill.
- **Welch's t-test** — tells how likely the difference is just luck (the **p-value**; below 0.05 means "not luck").
- **Bootstrap 95% CI** — a range that very likely contains the true improvement.

Real results

GAME	RANDOM	UNTRAINED	TRAINED	P-VALUE	95% CI OF GAIN
FrozenLake (goal reached)	1.2%	1.2%	**100%**	< 0.001	0.978 – 0.998
CartPole (steps balanced)	22.58	25.63	**425.92 ± 122.59**	0.0018	291.22 – 474.74



bar length = mean test score out of the 500-step maximum



Evaluation page — the verdict appears only when the gain is significant and its interval is above zero.

SECTION 15

15 Ablation study

An **ablation study** removes one part at a time to see how much that part matters — like baking a cake without sugar to learn what sugar does.

Full DQN
Replay memory and target network both on.

No replay
Learns only from the newest move.

No target network
The network sets its own targets.

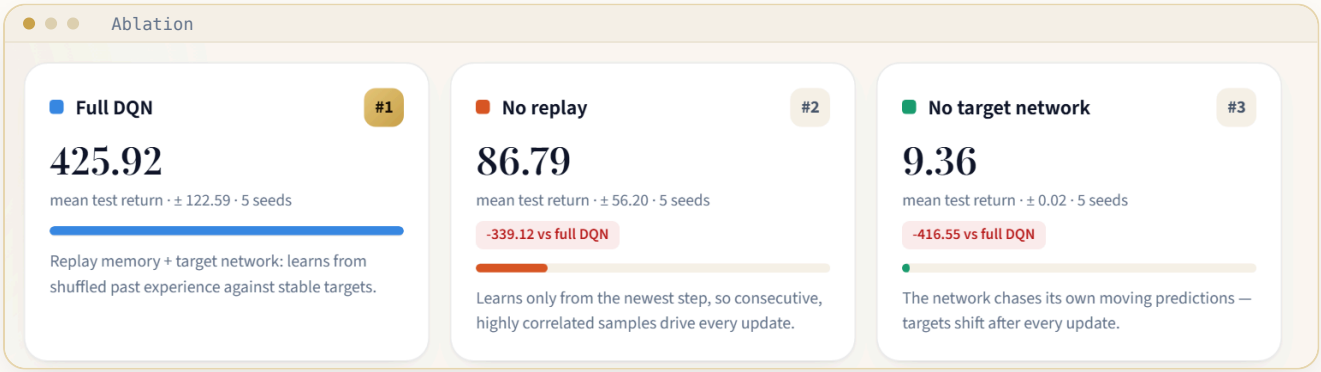
Real results (CartPole, 5 seeds each)

VARIANT	MEAN TEST SCORE	SPREAD
Full DQN	425.92	± 122.59
No replay	86.79	± 56.20
No target network	9.36	± 0.02

COMPARISON	GAIN	P-VALUE
Full vs no replay	+339.12	0.0017
Full vs no target	+416.55	0.0016

What it teaches

- **Both parts are needed.** Removing either made the agent clearly worse, and it is not luck ($p < 0.05$).
- **Without the target network** the score (9.36) was about the same as an untrained agent — learning never settled.
- **Without replay** it learned a little (86.79) but stayed far below the full DQN.
- These numbers come from one set of settings (600 episodes each).



Ablation page — ranked variant cards with each variant's gap to the full DQN.

SECTION 16

16 AI Reflection

The reflection layer writes a short plain-English explanation of a run. It **explains** learning; it does not cause it. It is designed so it cannot invent numbers.

1 · Facts

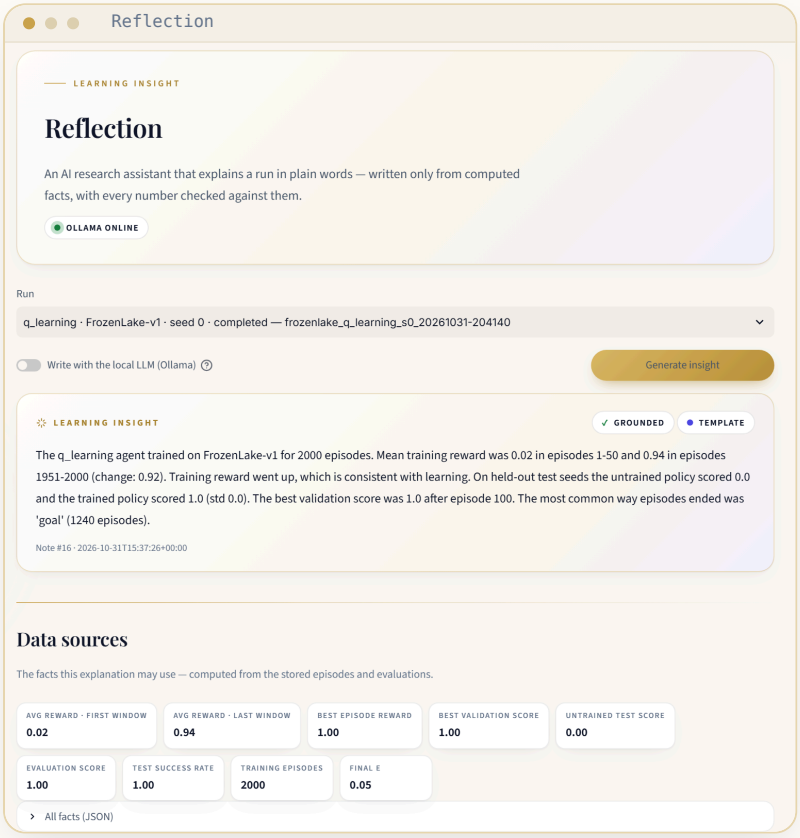
Numbers computed from the database: early and late average reward, best check score, before/after test score, random baseline, p-value, confidence interval, ε, loss.

2 · Write

A fixed **template** fills the facts into sentences. Optionally a small local AI (**Ollama**, model qwen2.5:1.5b) writes it instead — seeing only the facts.

3 · Check

Every number in the note must match a fact within 1%. If not, the AI note is **rejected** (kept for transparency) and the template note is shown.



Reflection page — choose a run, press Generate insight; the facts are shown as data sources.

Real example note

> "The dqn agent trained on CartPole-v1 for 600 episodes. Mean training reward was 21.98 in episodes 1-50 and 394.74 in episodes 551-600 … On held-out test seeds the untrained policy scored 9.38 and the trained policy scored 500.0 … Welch's t-test against random gave p = 0.0018."

- **Large Language Model (LLM)** — an AI that writes text.
- **Ollama** — a free tool to run a small LLM on your own computer.
- On the live website Ollama is not available, so the template is used.
- Users can rate each note: accurate yes/no and usefulness 1–5.

SECTION 17

17 Project file structure

How the folders are organised on GitHub.

```
Self-Learning-AI-Agent/
├── src/sla/                  # all program code (backend + frontend)
│   ├── envs/                 # games: FrozenLake, CartPole
│   ├── agents/               # random, Q-Learning, DQN, neural network, ε schedule
│   ├── memory/               # replay buffer for DQN
│   ├── training/             # config, runner (loop), callbacks, guards, pipeline
│   ├── evaluation/           # frozen tests, metrics, statistics, ablation, plots
│   ├── checkpoints/          # save / load agent snapshots
│   ├── database/             # SQLite store (8 tables)
│   ├── reflection/           # facts, template, Ollama client, number check
│   ├── services/             # shared actions for dashboard and CLI
│   ├── utils/                # errors, files, logging, seeds, validation
│   ├── app/                  # Streamlit dashboard
│   │   ├── main.py           # starts the website
│   │   ├── views/            # the 9 pages
│   │   ├── components/       # cards, badges, landing blocks, motion
│   │   ├── charts/           # Plotly figures
│   │   ├── styles/           # luxury theme and CSS motion
│   │   ├── data.py           # cached database reads
│   │   └── demo_seed.py      # loads results snapshot on the live site
│   ├── cli.py                # the "sla" command
│   └── settings.py           # paths for database and runs
├── configs/                  # YAML settings for every experiment
├── data/demo/                # read-only results snapshot for the live site
├── tests/                    # unit/, integration/, ui/ (pytest)
├── docs/                     # setup, usage, training, evaluation, NFRs, use cases
├── scripts/make_tables.py    # builds results tables
├── .github/workflows/ci.yml# automatic checks on every pull request
├── .streamlit/config.toml    # dashboard theme
├── requirements.txt          # packages to install
├── pyproject.toml            # project definition
├── README.md · CHANGELOG.md · LICENSE
└── runs/                     # (laptop only) sla.db + one folder per run
```

SECTIONS 18 & 19

18 How to run it · Testing & quality

Easiest way: open the live website — self-learning-ai-agent-final-year-project.streamlit.app. Nothing to install.

18 · Run it on your own computer

Needs Python 3.10 or newer. No GPU needed.

```
git clone <GitHub link on page 22>
cd Self-Learning-AI-Agent
python -m venv .venv
.venv\Scripts\activate     (Windows)
pip install -r requirements.txt
pip install -e .
sla init-db
sla dashboard
```

The dashboard opens at `http://localhost:8501`.

Using the website

1. Open **Overview** to see the status.
2. **Training** → choose FrozenLake → **Multi-seed experiment** → Start. Stay on the page until it finishes.
3. Open **Evaluation** and **Agent comparison** for the proof.
4. **Reflection** → Generate insight.

19 · Testing & quality

Automated tests are small programs that check the code works. The project has about **170 tests** written with **pytest**.

TYPE	WHAT IT CHECKS
Unit	Small parts alone: Q-update maths, replay buffer, statistics, database, number checker.
Integration	Whole flows: training + saving + resuming, full pipeline, a short DQN run.
UI	Every dashboard page opens — with an empty database and with real data.

- **Ruff** checks code style and common mistakes.
- **GitHub Actions (CI)** runs Ruff and the tests automatically on every pull request and every push to main.
- Every run stores its seed, settings and code version, so results can be **repeated**.

SECTIONS 20 & 21

20 Challenges & limitations

Real problems we met while building and deploying the project, and how we solved them.

CHALLENGE	HOW WE SOLVED IT
DQN became unstable	Added a target network, Huber loss and gradient clipping; the ablation proves the target network is essential.
Training scores look better or worse than reality	Separate frozen tests on new seeds, 5 seeds, and a random baseline.
Live site crashed: “module not found”	The app now adds its own source folder to Python's search path at start-up.
PyTorch would not install on Streamlit Cloud	Linked the small CPU-only PyTorch file for each Python version.
Streamlit showed extra unwanted pages	Renamed the pages folder to views/ so Streamlit stops auto-loading it.
Experiments stopped half-way on the website	Training runs inside the open page; added a clear warning and made the live panel faster.
Live panel mixed all 5 seeds	The panel now resets for each seed and shows “run 2 / 5”.
Live site started with no results	A read-only snapshot of real results is loaded when the hosted app starts.

21 · Limitations (honest)

- Only two small benchmark games; the agent learns one task at a time.
- DQN results depend on settings and seeds; 5 seeds give wide confidence intervals.
- Training in the browser stops if the page is closed (no background worker yet).
- The live website's database is temporary and resets when the app restarts.
- The number checker verifies values, not meaning — the template note stays the reference.
- CPU only: bigger environments would be slow.

SECTION 22

22 Future scope & add-ons

Ideas to grow the project. **None of these are implemented yet** — they are planned or possible next steps.

Double DQN NOT YET
Uses two networks to avoid over-estimating action scores.

Dueling DQN NOT YET
Splits the network into "how good is this state" and "how good is each action".

Prioritized Experience Replay NOT YET
Replays the most surprising memories more often.

PPO / A2C agents NOT YET
Policy-gradient methods that learn actions directly.

Harder games NOT YET
LunarLander, MountainCar, Atari games.

Slippery & 8×8 FrozenLake NOT YET
Random slips and bigger maps make the puzzle harder.

Background training worker NOT YET
Training continues after the browser tab is closed.

Hyper-parameter search (Optuna) NOT YET
Automatically finds better settings.

GPU support NOT YET
Faster training for bigger networks.

Cloud database (PostgreSQL / Supabase) NOT YET
Permanent storage for the live site.

User login & saved experiments NOT YET
Each user keeps their own runs.

Gameplay replay video / GIF NOT YET
Watch the trained agent play.

Live leaderboard NOT YET
Rank agents and settings by test score.

LLM hyper-parameter advisor NOT YET
Reflection notes suggest new settings to try.

REST API (FastAPI) NOT YET
Other apps can start runs and read results.

Docker deployment NOT YET
One-command setup anywhere.

Mobile-friendly UI NOT YET
Comfortable use on phones.

SECTION 23

23 Glossary

Every technical word in one simple line.

Agent	The learner that makes decisions.
State	What the agent sees right now.
Reward	Points given after a move.
Policy	The agent's habit: which action it picks in each state.
Q-Learning	Learning a table of Q-values from rewards.
Neural network	A function with many adjustable numbers that learns from examples.
Replay memory	Storage of past moves used for learning.
ε (epsilon)	The chance of taking a random action to explore.
γ (discount)	How much future rewards count.
Checkpoint	A saved snapshot of the agent.
Baseline	A simple player to compare against.
Confidence interval	Range that very likely contains the true value.
SQLite	A database stored in one file.
Plotly	A library for interactive charts.
Gymnasium	A library of training games for RL.
YAML	A simple text format for settings files.

Environment	The game or world the agent acts in.
Action	A move the agent can make.
Episode	One full game from start to end.
Q-value	Score for “how good is this action in this state”.
DQN	Q-Learning where a neural network estimates the Q-values.
Weights	The adjustable numbers inside a neural network.
Target network	A frozen copy of the network that gives steady targets.
α (learning rate)	How big each learning step is.
Loss	How wrong the network's predictions were.
Seed	A number that fixes randomness so runs repeat.
p-value	Chance the difference is just luck; < 0.05 = significant.
Ablation	Removing a part to see how much it matters.
Streamlit	A Python tool that makes web dashboards.
PyTorch	A library for building neural networks.
CLI	Command-line interface: typing commands instead of clicking.
CI	Automatic checks run on every code change.

SECTION 24

24 Conclusion & quick summary

We built an AI agent that starts with no knowledge, learns two tasks only from rewards, and proves its learning with fair tests and statistics — all shown in a public dashboard.

WHAT	A reinforcement-learning system: Q-Learning on FrozenLake, a from-scratch DQN on CartPole.
HOW IT LEARNS	Observe → act → reward → update the table or network, thousands of times, exploring less over time.
HOW WE PROVE IT	Frozen agent, 100 new games, 5 seeds, vs untrained and random, Welch's t-test and 95% CI.
RESULTS	FrozenLake **100%** vs random 1.2%. CartPole **425.92** vs random 22.58 (p = 0.0018). Ablation: full 425.92, no replay 86.79, no target 9.36.
STACK	Python, PyTorch, Gymnasium, SQLite, Streamlit, Plotly, pytest, GitHub Actions.
WHY IT MATTERS	It shows the difference between an AI that only talks and one that **improves from experience** — and how to prove it honestly.

GITHUB

https://github.com/brahmanandmathpati/Self-Learning-AI-Agent

LIVE DASHBOARD

https://self-learning-ai-agent-final-year-project.streamlit.app/

IN ONE SENTENCE

The agent is a student who starts knowing nothing, practises with rewards, and passes a fair exam it has never seen — and we can show every mark it earned.

VIVA

25 Viva questions & answers

Ten questions an examiner is likely to ask, with short simple answers.

Q1. What is reinforcement learning?
Teaching an agent with rewards: it tries actions and repeats the ones that earn points.

Q2. Why is this different from a chatbot?
A chatbot answers from fixed knowledge; our agent changes its own numbers from experience and gets better scores.

Q3. Why two algorithms?
FrozenLake has 16 states, so a table works (Q-Learning). CartPole has endless states, so we need a neural network (DQN).

Q4. What is ε-greedy?
Mostly pick the best known action, but with chance ε pick randomly to explore; ε falls from 1.0 to 0.05.

Q5. Why replay memory and a target network?
They make DQN stable. Our ablation shows scores drop to 86.79 and 9.36 without them.

Q6. How do you know the agent really learned?
Frozen tests on new games, 5 seeds, against untrained and random players, with p = 0.0018 and a CI above zero.

Q7. What is a p-value?
The chance the difference happened by luck; below 0.05 we treat it as real.

Q8. Why SQLite?
Free, one file, no server, built into Python, and fast enough for our 20,314 episodes.

Q9. Can the reflection AI make up numbers?
No. Every number is checked against database facts; a note with a wrong number is rejected.

Q10. What would you add next?
Double/Dueling DQN, harder games, a background training worker and a permanent cloud database.